# Cone detection for Formula Student images

Nicola Ranzolin Edin Milenko Alessandro Martini

September 15, 2026

## 1 The task and the data

Formula Student cars drive on a track delimited by cones: blue on the left, yellow on the right, orange (small or large) around the start and stop areas. The task is to detect every cone in an image, assign it to one of the four classes and segment it. The images come from the FSOCO dataset, collected by many teams with different cameras and lighting. The annotations are in one JSON file per image. Merging the two archives provided, one with bounding boxes and one with masks, gives 5755 training images with 112 674 cone boxes. The test set is a separate folder of 24 images that also appear in the training data under the same names, so they are explicitly skipped when the training set is built: none of them is ever seen during training.

## 2 Overview

The pipeline runs five steps on every image:

1. **Candidate generation.** Cones are either blue, yellow or orange, a property almost nothing else on a track has, so instead of sliding a window over every position and size, a colour mask marks the pixels that could belong to a cone and a fixed set of boxes(candidates) is generated around each colour blob.

2. **Two-stage classification.** Every candidate is resized to a small patch and described by HOG on its grey version, which captures the shape, plus a hue and saturation histogram, which puts back the colour HOG throws away. The descriptor is then fed to a linear SVM with a permissive threshold to reject most of the obvious negative candidates, and an SVM with an RBF kernel, trained only on what the first one accepts, decides which of the survivors are cones.

3. **Colour classification.** Each survivor from the two-stage gets a class by counting coloured pixels inside its box. Small and large orange cones share the same colour, so they are told apart by counting the white stripes. The colour chosen is the one that dominates the other two.

4. **Segmentation.** The box is padded and its pixels clustered with k-means in HSV; the cluster voted by the colour mask of the predicted class is the cone, and a morphological closing operator glues back the pieces that the white stripes cut apart.

5. **Final box.** The boxes provided by the two-stage are very permissive on purpose, so the final bounding box is the one that contains the segmentation mask, not the initial candidate.

## 3 Candidate generation

Searching every position and every size would mean millions of windows per image. Cones however have a property almost nothing else on a track has: they are saturated blue, yellow or orange, so the search is restricted to where those colours appear.

The image is first white balanced with a "shades of grey" correction: for each channel the mean of the intensities raised to the sixth power is computed, its sixth root is taken, and the channels are rescaled so that the three values become equal. The high power makes the estimate depend on the brightest pixels. This correction works very well in practice since the images of the dataset have very different lighting.

The corrected image is converted to HSV and three hue ranges are kept, blue (100–130), orange (5–20) and yellow (20–35), all with saturation at least 80 and value at least 60. The floors are high on purpose: with lower ones dry grass enters the yellow range and merges with the cones into useless blobs. Components under 3 pixels are discarded since they are noise.

The bounding box of a blob would be the natural candidate, but it does not work well in practice: the stripes split a cone into several blobs, so each blob gives only a rough position of the cone and its size says almost nothing about the cone's size. Instead, seven boxes are generated for each color blob, all centered on it, with fixed heights of 10, 18, 30, 50, 85, 140 and 220 pixels and a 2:3 ratio between width and height. The price is volume and precision: over 20k candidates per image, which is what forces the next step to be a cascade.

## 4 Two-stage classification

With over twenty thousand candidates and around sixteen real cones per image, a single classifier would not be powerful enough. But a linear classifier is cheap, so it is used first with a permissive threshold to throw away most of the candidates, and a more expensive non-linear one looks only at what is left. On the test images: 21 709 candidates per image, 1531 after the first stage, 43 after the second, 12 after merging overlaps (Table 2). This step is only used to find cones in the image. Once the box is found, identifying the color is a much easier task, done later.

**Discarded approach.** Viola-Jones was considered as a first candidate detector because of its low computational cost and simple cascade structure. Training one is no longer possible with a recent OpenCV, `opencv_createsamples` and `opencv_traincascade` were dropped after the 4.0 version, and only the detector class remains, so we trained the cascades with an older version of the library. Several Haar and LBP cascades were tested, including hard-negative mining and a custom non-maximum suppression, but the best configuration still performed very poorly, with many false positives coming from background structures. HOG and an SVM did substantially better on the same task, so the cascade was dropped and the work went on with the latter.

**Descriptor.** Every candidate is resized to a $16\times24$ patch, with area interpolation when shrinking and linear interpolation when enlarging. The descriptor is a HOG on the grey patch, followed by a $12\times4$ hue-saturation histogram. The histogram is there because HOG throws the colour away, and colour is exactly what separates a cone from a piece of asphalt with the same triangular shape.

**Stage one.** A linear SVM trained by averaged stochastic gradient descent with a soft margin. The aim of this one is not to decide but to keep the true cones and drop as much as possible. At this stage early false rejection is fatal.

**Stage two.** An SVM with an RBF kernel, trained only on the patches stage one lets through, so that it learns the distribution it will actually see.

**Non-maximum suppression.** Since a cone produces several blobs and each blob seven boxes, the same cone is found many times. Overlaps are merged at an IoU of 0.1, a very low threshold that works decently well in practice since real cones almost never overlap in the image.

**Training.** Since our approach generates candidates that are not perfectly around the cone and that include a high percentage of the background, a classifier trained on clean patches of the annotated boxes didn't work well. So the training patches are the candidates themselves, produced by running the candidate generation described before on the training images and labelled by their IoU with the annotations: at least 0.3 makes a positive, below 0.2 a negative. Positives are flipped horizontally, which doubles them(very simple data augmentation). Negatives are capped at 50 per image and positives are not: with the classes even, stage one scored everything low and the cascade stopped detecting. After a first training, stage one is run on up to 400 training images and the candidates it wrongly accepts (IoU below 0.2) are added to the negatives, and stage one is retrained; this technique is the hard negative mining mentioned in class. Stage two is then trained on the survivors of stage one, at most 5000 per class, balanced, the cap being there because an RBF SVM gets slow with many samples.

## 5 Colour classification

Once a cone is detected, the color is then found separately without training any model. The value channel of each box is equalized with CLAHE(a version of local histogram equalization), which brightens the cones in shadow without saturating the ones in the sun. Three HSV masks are computed, yellow, blue and orange, restricted to a trapezoid that cuts the upper corners of the box by a quarter of its width each: a cone is triangular, and those corners are always background. Yellow or blue is chosen only when it beats both other colours; otherwise the cone is orange. Small and large orange cones have the same colour and are told apart by their white stripes, two on the large one. A white mask is cleaned with a horizontal opening and its blobs counted, keeping those larger than 0.8% of the box. Two or more stripes mean a large cone.

## 6 Segmentation and final box

The box received by the classification is first padded 40% of the height at the bottom and 10% of the width on the sides. This was added since the imprecise bounding boxes often cut the bottom part of the cones. Boxes shorter than 80 pixels are upscaled before segmenting, since a tiny cone has too few pixels for clustering and morphology, and the mask is scaled back at the end. The pixels are clustered with k-means in HSV with $K = 3$, the natural number of things in the box: background, cone colour and white stripes. Each cluster receives one vote per pixel of the colour mask of the predicted class, and the winner is kept. The coloured cluster is cut into pieces by the stripes, so a closing with a tall thin kernel glues them back. The largest connected component is kept and its outer contour filled, which closes the stripes left as holes. The mask follows the cone much better than the padded box, so the reported box is the bounding box of the mask. If no mask was found, the padded box is kept.

## 7 Evaluation

The ground truth box of a cone is the extent of its annotated mask. For the **macro-averaged F1**, the detections of each class are sorted by score and matched greedily to the ground truth cones of the same image and class at IoU $\geq 0.5$, each cone matched at most once; the four per-class F1 values are averaged without weights, so the 32 orange cones weigh as much as the 366 yellow and blue ones. For the **mIoU**, label maps of predictions and ground truth are compared pixel by pixel: for each class, true positive, false positive and false negative pixels are accumulated over the whole test set, the Jaccard index $\mathrm{TP}/(\mathrm{TP} + \mathrm{FP} + \mathrm{FN})$ is computed and the four values averaged. This is a metric over the whole image, so it pays for every missed cone and every false detection.

Table 1: Results on the test set (24 images, 398 cones).

| Metric | Value |
|---|---|
| Macro-averaged F1 (IoU $\geq 0.5$) | 0.543 |
| mAP@0.5:0.95 | 0.165 |
| mIoU (pixels) | 0.294 |

Table 2: Windows surviving each step, averaged over the test images.

| Step | Windows per image |
|---|---|
| Candidates from the colour mask | 21 709 |
| After the linear stage | 1 531 |
| After the RBF stage | 43 |
| After non-maximum suppression | 12 |

## 8 Results

Table 1 gives the three metrics, and Appendix shows the output and the metrics on each test image. The results vary a lot from image to image, with an F1 ranging from 0 on the image taken against the sun to 0.909 on the best one.

## 9 Limitations

The scores are modest, and the reasons are various.

The first limit is in the candidate step. Its boxes come from seven fixed heights centred on a colour blob, so they are roughly right but rarely tight and can still miss some parts of the cones. The mask can still improve the box afterwards, but only for the cones that get through the classifier.

The second limit is the size of the cones. Half of them are under 36 pixels tall and a quarter under 23. Every candidate is resized to a $16\times24$ patch, so a small cone is upscaled and blurred before HOG sees it, and a descriptor of a few hundred numbers has little to work with.

The third limit is localization. The final box is the bounding box of a mask produced by clustering, which is not always extremely precise.

The metrics themselves amplify this. The macro average gives half of the F1 to two classes with 14 and 18 cones, where a single cone moves the F1 of its class by several hundredths, and the pixel mIoU pays for every missed cone and every false detection, not only for the quality of the masks that were found.

The fourth limit is execution speed. The project specification requires the algorithm to run in real-time on an onboard vehicle computer, but the pipeline on average runs at around 1 image per second. The time is spent almost entirely on the candidate evaluation: computing HOG features for over 20k patches per image and evaluating the RBF SVM on over 1500 of them takes the vast majority of the processing time.

Finally, the detector is a pair of SVMs on hand-made features, the second one trained on at most 5000 patches per class. It does not have the capacity, nor the data, to learn the variety of backgrounds, lighting and cone sizes of these images. A possible upgrade could be adding a third/forth processing stage, which would require weeks of work and would also increase the processing time even more.

A lesson was learned along the way: a classifier has to be trained on the same kind of input it will see, not on the perfect and ideal candidates. Additionally the trained models are shipped with the code; first because they are not heavy, secondly because training the SVM on different versions of opencv may change very slightly the metrics of the trained models, but running eval with the models provided will produce the same numbers provided in the table.

# 10 Team Contributions

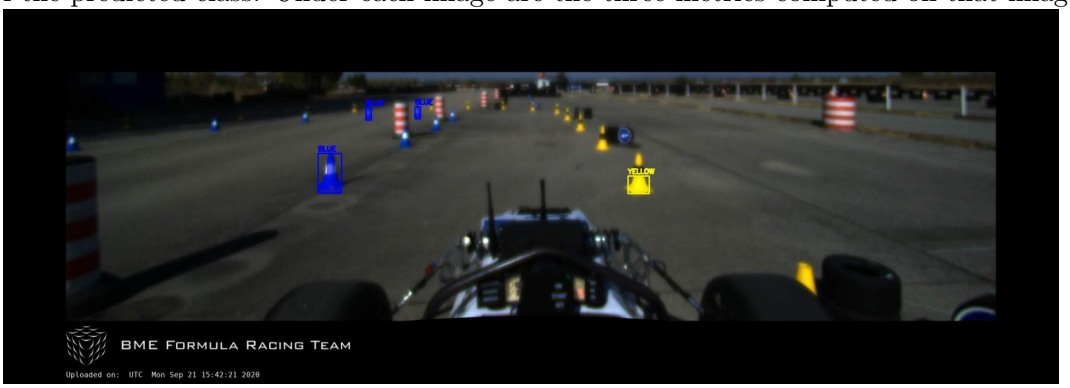
The following list details the specific contribution of each member in terms of ideas, implementation, tests, performance measurement and total working hours.

## Contributions

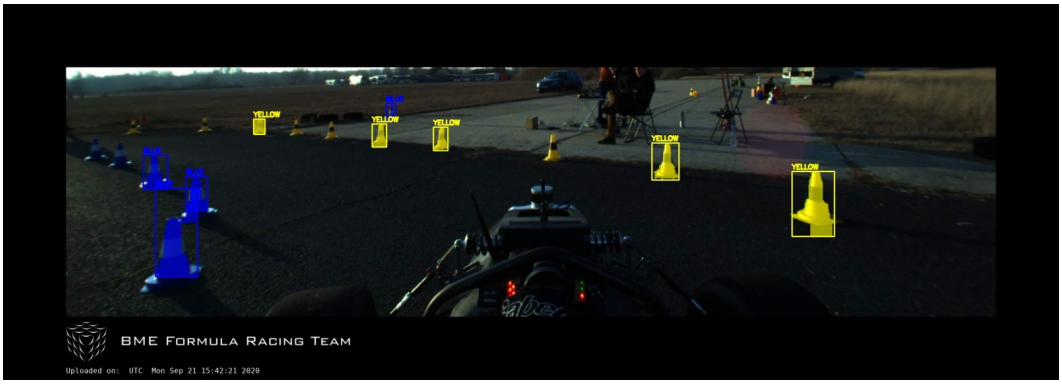
1. **Nicola Ranzolin** - 120 hours
    - All files with author: `dataset` (hpp and cpp), `patches` (hpp and cpp), `detector` (hpp and cpp).
    - Ideas used: Viola-Jones style detector as the starting point, hard negative mining to build the negative patches.
    - Dataset preprocessing and parsing: merging the two FSOCO archives, decoding the Supervisely bitmap masks, deriving the bounding boxes from the masks.
    - Patch extraction from the candidates and assembly of the final pipeline.
    - Research on object detection and on the FSOCO annotation format.
    - Testing on: proposal patches versus clean crops of the annotated boxes for training, balanced versus capped negatives, the IoU thresholds that label a candidate as positive or negative and the non-maximum suppression threshold.
2. **Edin Milenko** - 125 hours
    - All files with author: `classifier` (hpp and cpp), `segmentation` (hpp and cpp), `utils` (hpp and cpp).
    - Ideas used: CLAHE on the value channel, HSV colour masks, trapezoid mask to cut the background corners, k-means clustering for segmentation, white stripe counting to tell small and large orange cones apart.
    - Implementation of the three metrics required by the specification: macro-averaged F1, mAP@0.5:0.95 and pixel mIoU.
    - Research on segmentation and on colour spaces.
    - Testing on: rejecting boxes with too few coloured pixels, number of k-means clusters, size of the closing kernel, the upscaling threshold before segmenting, and widening the padding to give the segmentation more context.
3. **Alessandro Martini** - 123 hours
    - All files with author: `hog` (hpp and cpp), `color_proposals` (hpp and cpp), `main` (cpp).
    - Ideas used: colour proposals to prune the search space, two-stage classification handcrafted as a cascade (linear SVM followed by an RBF SVM), HOG combined with a hue-saturation histogram as the descriptor.
    - Implementation of the descriptor and of the training of both classifiers.
    - Research on HOG, on SVM training and on cascade classifiers.
    - Testing on: weight of the colour histogram next to the HOG values, saturation and value floors of the colour mask for the color proposal, the set of proposal heights, the thresholds of both cascade stages, and the regularization of the RBF stage (the value of C)

# A Output on the test set

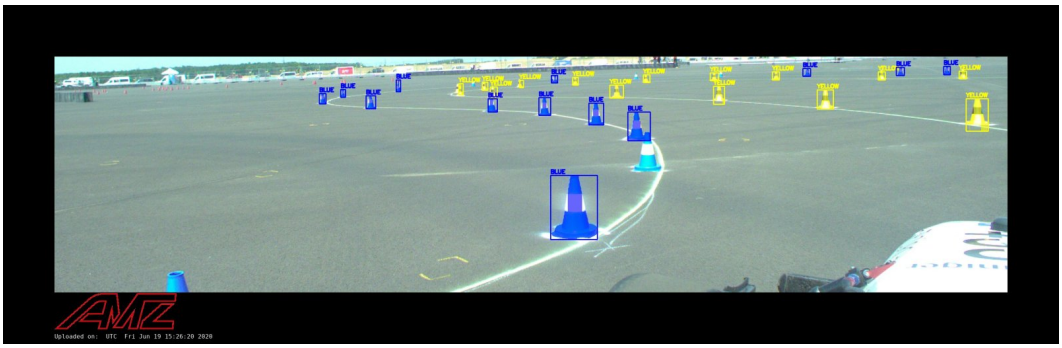
Every image of the test set, with the box, the label and the mask of each detected cone drawn in the colour of the predicted class. Under each image are the three metrics computed on that image alone.



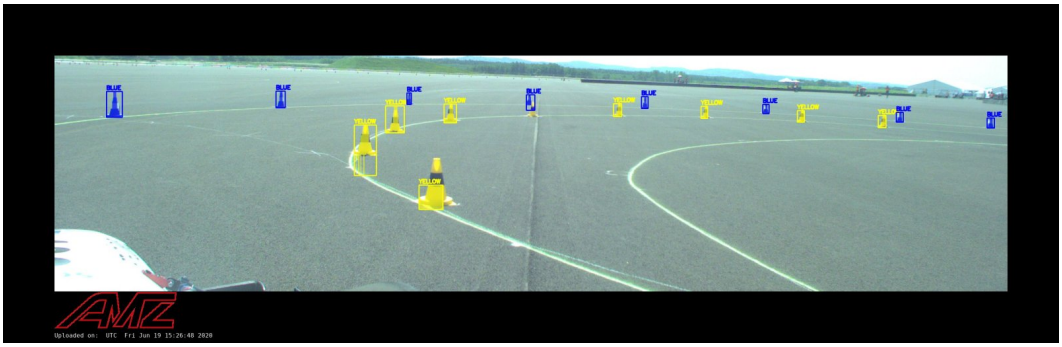

F1 0.273 mAP 0.5:0.95 0.091 mIoU 0.288



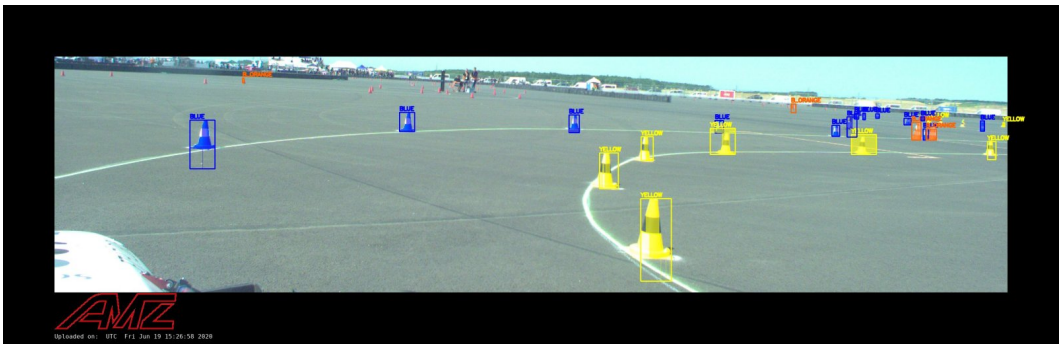

F1 0.533 mAP 0.5:0.95 0.166 mIoU 0.544



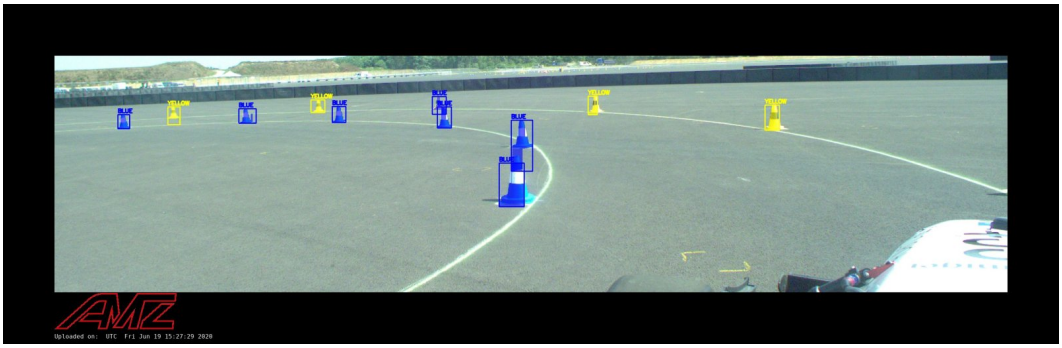
F1 0.780 mAP 0.5:0.95 0.310 mIoU 0.623



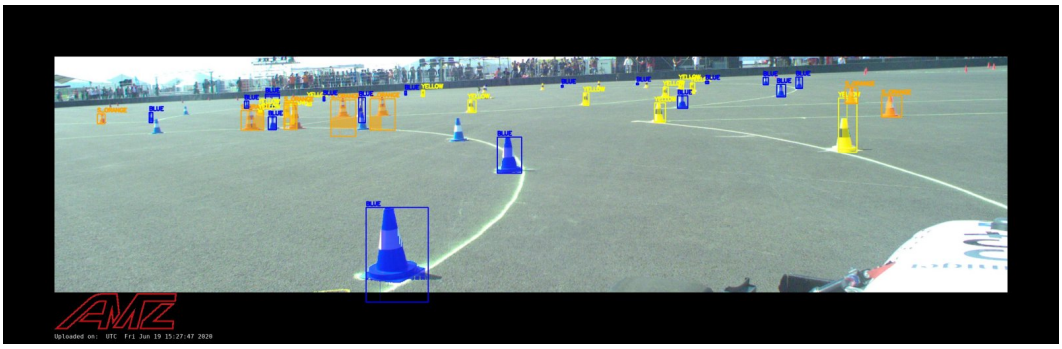
F1 0.849 mAP 0.5:0.95 0.310 mIoU 0.593



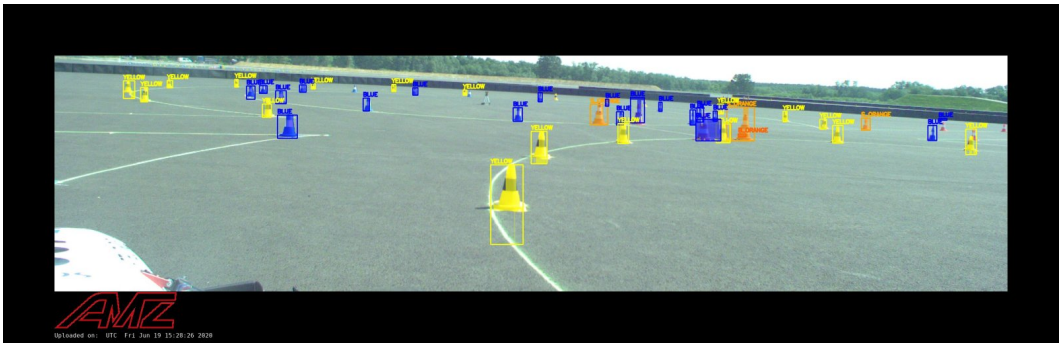
F1 0.694 mAP 0.5:0.95 0.259 mIoU 0.532



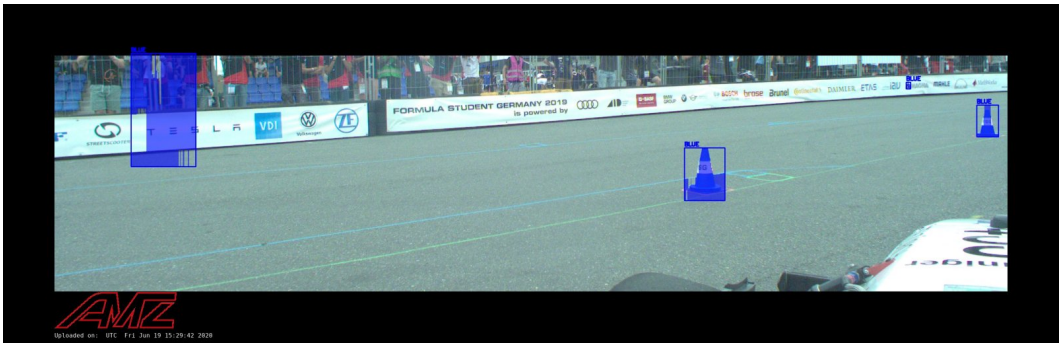
F1 0.795 mAP 0.5:0.95 0.264 mIoU 0.621



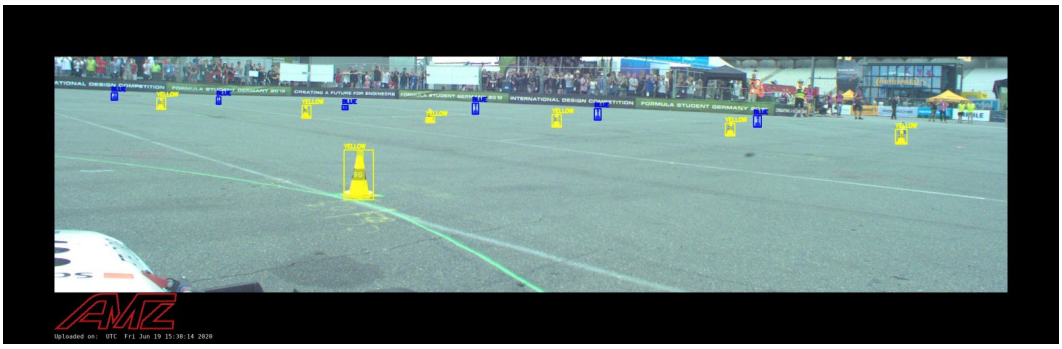
F1 0.414 mAP 0.5:0.95 0.106 mIoU 0.338



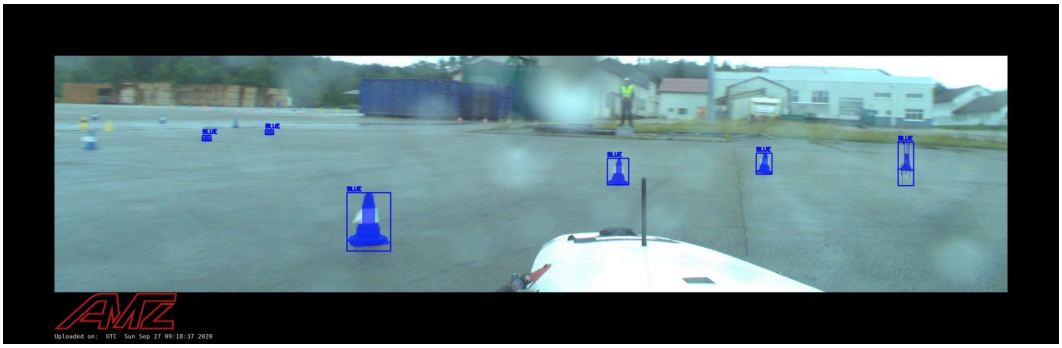
F1 0.441 mAP 0.5:0.95 0.101 mIoU 0.293



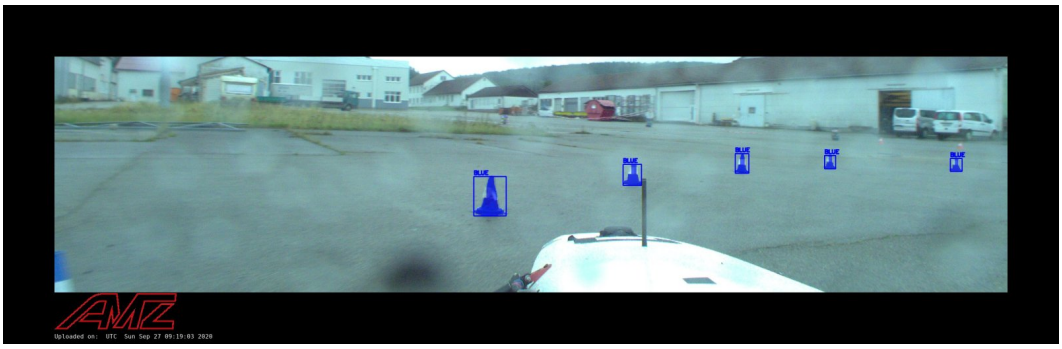

F1 0.667 mAP 0.5:0.95 0.625 mIoU 0.126



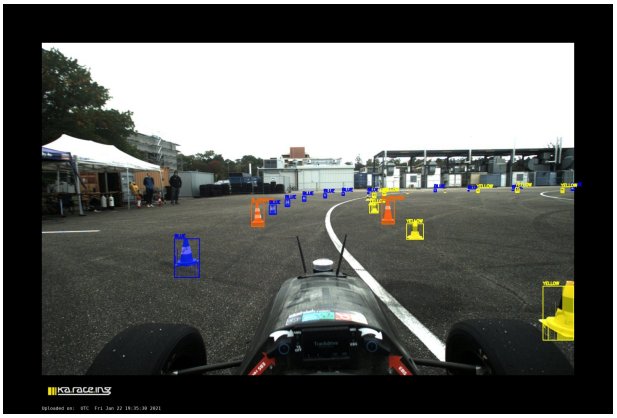
F1 0.736 mAP 0.5:0.95 0.434 mIoU 0.646



F1 0.187 mAP 0.5:0.95 0.116 mIoU 0.316



F1 0.909 mAP 0.5:0.95 0.400 mIoU 0.474



F1 0.832 mAP 0.5:0.95 0.433 mIoU 0.710



F1 0.382 mAP 0.5:0.95 0.117 mIoU 0.464



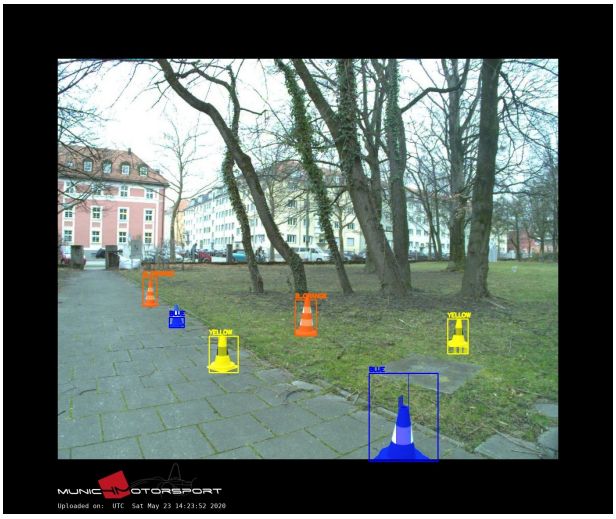
F1 0.875 mAP 0.5:0.95 0.342 mIoU 0.551



F1 0.400 mAP 0.5:0.95 0.051 mIoU 0.329



F1 0.000 mAP 0.5:0.95 0.000 mIoU 0.000



F1 0.833 mAP 0.5:0.95 0.459 mIoU 0.789



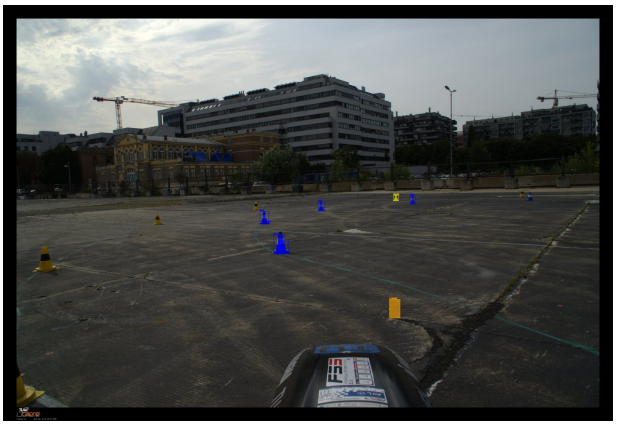
F1 0.536 mAP 0.5:0.95 0.234 mIoU 0.269



F1 0.345 mAP 0.5:0.95 0.132 mIoU 0.413



F1 0.536 mAP 0.5:0.95 0.289 mIoU 0.138



F1 0.439 mAP 0.5:0.95 0.206 mIoU 0.256



F1 0.842 mAP 0.5:0.95 0.505 mIoU 0.411



F1 0.593 mAP 0.5:0.95 0.382 mIoU 0.480